The Journal of Open Source Software

Baar-Core: Pre-flight budget enforcement and budget-aware model routing for LLM agents

Robat Das [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 06 November 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Applications built on large language models (LLMs) are billed per token, and autonomous agents issue many calls whose number and length are not known in advance. A looping or misbehaving agent can therefore spend far more than its operator intended before anyone notices. Baar-Core is a Python library that enforces spending limits *before* each LLM request is sent. It estimates the cost of a call locally, reserves that amount against a budget, and raises an exception without making any network request if the call cannot be afforded. On top of this enforcement layer, Baar-Core routes each task to a cheaper or more capable model according to an estimate of task complexity and the budget remaining, downgrading to the cheaper model when the budget runs low so that work can still complete. Two certified decisions build on this layer: a `calibrated` reservation mode that bounds the probability that a whole agent run exceeds its budget, and an early-stopping policy that stops runs that cannot finish within budget while bounding how many runs that would have succeeded are stopped. It works with any provider supported by LiteLLM (BerriAI, 2023) and integrates with LangChain, LangGraph, CrewAI, Hermes, the Vercel AI SDK and other agent frameworks.

Statement of need

Researchers who run LLM agents at scale, for benchmark evaluation, agent experiments or user studies, need costs that are bounded and reproducible. Measurement studies show that agent token consumption is highly variable: repeated runs of the same coding task can differ by up to 30x in total tokens, and agents predict their own costs poorly (Bai et al., 2026; Lin et al., 2026). Unbounded resource consumption is also listed as a top-ten risk for LLM applications (OWASP Foundation, 2025). Most existing tooling reports spend after the fact, through dashboards and alerts, or enforces limits at a hosted proxy. Baar-Core targets researchers and developers who need a hard, local, in-process limit: a budget that a single experiment, agent run, user or time window cannot exceed, enforced without an external service.

State of the field

LiteLLM (BerriAI, 2023) provides budget tracking for its proxy server and SDK. Hosted gateways such as Portkey and Helicone offer spend monitoring and rate limits. These tools are oriented around observability, or around enforcement at a network proxy, rather than an in-process pre-flight decision that also accounts for the uncertain length of the model's output. Token Budgets (Khan, 2026) catalogs real budget-overrun incidents in agent frameworks and provides a reservation-based cap in Rust. Green SARC (Besanson, 2026) proposes predictive cost governance with conformal bounds, implemented as a sidecar. Research on LLM routing and cascades, such as FrugalGPT (Chen et al., 2023), RouteLLM (Ong et al., 2024) and C3PO (Valkanas et al., 2025), studies how to trade cost against quality, typically evaluated in

DRAFT

aggregate rather than as a per-run hard limit.

We built a standalone library rather than contributing to LiteLLM because the enforcement decision has to wrap routing, downgrading and framework callbacks together, and has to run without a proxy. Baar-Core uses LiteLLM as its provider layer and pricing source instead of re-implementing it.

Software design

The core is a `BudgetTracker` that holds the budget as the single source of truth. Before a call, the router computes an estimated cost from the prompt's token count and a planned output length, and *atomically reserves* that amount. After the call it reconciles the reservation with the cost the provider actually reported. Atomic reservation prevents concurrent asynchronous calls from each passing the check and jointly overspending.

The main design trade-off is how to plan for output length, which is unknown until generation ends. Baar-Core makes this an explicit, user-selected mode rather than a hidden heuristic:

- `estimate` reserves the expected output. Responses are never cut off, but a long response can overshoot the budget.
- `preflight` reserves the model's worst-case output. It never overshoots and never truncates, at the price of conservative admission.
- `truncate` (opt-in) derives the provider's `max_tokens` from the remaining budget, so a long response is cut off instead of overshooting; `continue` generates the answer in budgeted segments.
- `calibrated` reserves a quantile of logged output lengths whose level is certified on whole runs with fixed-sequence Learn-then-Test (Angelopoulos et al., 2021), so that a run overspends with probability at most a chosen ε (Das, 2026). The per-call alternative, `overrun_risk`, reserves a split-conformal quantile per call.

Silent truncation is never the default, because a truncated answer can corrupt an experiment without any visible error.

Budgets can persist through pluggable stores (in-memory, file, SQLite), and can reset over hourly, daily or monthly windows for per-user quotas. A rule-based `Policy` layer can reject or reroute tasks. Routing uses a small model as a complexity scorer. When the capable model is unaffordable, the router downgrades instead of failing. Every decision is recorded in a routing log for audit. Framework integrations are thin adapters over this one enforcement path, so the same guarantees apply whether calls come from LangChain callbacks, LangGraph middleware, CrewAI step callbacks or tool wrappers. A benchmark harness (`baar-bench`) and a telemetry summarizer (`baar-telemetry`) support reproducible cost experiments.

For long-horizon agents, `EarlyStopPolicy` fits a per-checkpoint predictor of budget overrun on logged runs and certifies its stopping threshold by replaying calibration runs; the router checks it before every call and ends the run before any request is sent. The `baar.calibration` and `baar.early_stop` modules are dependency-free. Tool schemas are counted in pre-flight prompt estimates, and an OpenAI-compatible proxy forwards tool calls, so agents that use function calling keep the same guarantees.

Research impact statement

Baar-Core is the experimental platform and reference implementation of a research study on certified budget decisions for LLM agents (Das, 2026). The study evaluates the library's controller on 2,100 real gpt-4o-mini and gpt-4o agent runs and on 3,000 public OpenHands trajectories (Wang et al., 2024), and its findings fed back into the library: it exposed a prompt-counting error for tool-using calls and a proxy that dropped tool definitions, both

DRAFT

fixed in release 0.11.0, and it introduced the `calibrated` mode (0.11.0) and certified early stopping (0.12.0). All code, logged runs and analysis scripts for the study are in the repository. Baar-Core is distributed on PyPI, where it was downloaded about 2,000 times in its first six months (excluding mirrors), and is developed in the open with tagged releases, a changelog, continuous integration on Python 3.10–3.12, coverage reporting and more than 1,100 tests.

AI usage disclosure

Generative AI tools were used in developing this software and in preparing this paper: Claude Sonnet 4.6 and Claude Opus 5.5 (Anthropic), used through Claude Code, as recorded in the repository's commit history. They were used for code generation, tests, documentation, the research analysis code and drafting of this paper. The author made the architectural and design decisions, including the pre-flight reservation model, the explicit output-planning modes and the certified run-level methods. The author reviewed, modified and tested all AI-generated code and text. Correctness is checked by the automated test suite run in continuous integration.

Competing interests

The author also develops noburn.dev, a commercial service built on Baar-Core. Baar-Core itself is and remains open source under the MIT license.

Acknowledgements

This work received no external funding.

References

Angelopoulos, A. N., Bates, S., Candès, E. J., Jordan, M. I., & Lei, L. (2021). *Learn then test: Calibrating predictive algorithms to achieve risk control*. https://doi.org/10.48550/arXiv.2110.01052

Bai, L., Huang, Z., Wang, X., Sun, J., Mihalcea, R., Brynjolfsson, E., Pentland, A., & Pei, J. (2026). *How do AI agents spend your money? Analyzing and predicting token consumption in agentic coding tasks*. https://doi.org/10.48550/arXiv.2604.22750

BerriAI. (2023). *LiteLLM: Python SDK and proxy server to call LLM APIs in the OpenAI format*. https://github.com/BerriAI/litellm

Besanson, G. (2026). *Green SARC: Predictive cost and carbon governance for agentic AI systems*. https://doi.org/10.48550/arXiv.2606.15954

Chen, L., Zaharia, M., & Zou, J. (2023). *FrugalGPT: How to use large language models while reducing cost and improving performance*. https://doi.org/10.48550/arXiv.2305.05176

Das, R. (2026). *How often will my agent overspend? Certified run-level budget decisions for LLM agents*. Preprint; Zenodo. https://doi.org/10.5281/zenodo.23992231

Khan, S. (2026). *Token budgets: An empirical catalog of 63 LLM-agent budget-overrun incidents, with an affine-typed Rust mitigation as a case study*. https://doi.org/10.48550/arXiv.2606.04056

Lin, Y., Wang, Z., Liu, M., Shan, Y., Bai, L., Zhang, J., Jin, X., Chen, B., Su, J., Wang, X., Pei, J., & Li, M. (2026). *BAGEN: Are LLM agents budget-aware?* https://doi.org/10.48550/arXiv.2606.00198

DRAFT

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., & Stoica, I. (2024). *RouteLLM: Learning to route LLMs with preference data*. https://doi.org/10.48550/arXiv.2406.18665

OWASP Foundation. (2025). *OWASP top 10 for LLM applications 2025: LLM10 unbounded consumption*. https://genai.owasp.org/llmrisk/llm102025-unbounded-consumption/

Valkanas, A., Pal, S., Rumiantsev, P., Zhang, Y., & Coates, M. (2025). C3PO: Optimized large language model cascades with probabilistic cost constraints for reasoning. *Advances in Neural Information Processing Systems*. https://arxiv.org/abs/2511.07396

Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., & others. (2024). *OpenHands: An open platform for AI software developers as generalist agents*. https://doi.org/10.48550/arXiv.2407.16741

DRAFT